Symbol Table Management

Symbol table is an important data structure created and maintained by compilers in order to store information about the occurrence of various entities such as variable names, function names, objects, classes, interfaces, etc. Symbol table is used by both the analysis and the synthesis parts of a compiler.

A symbol table may serve the following purposes depending upon the language in hand:

- To store the symbol names of all entities in a structured form at one place.
- To verify if a variable has been declared.
- To implement type checking, by verifying assignments and expressions in the source code are semantically correct.
- To determine the scope of a name (scope resolution).
- To store the attribute of a symbol. An attribute for a symbol in the source code is the information associated with that symbol. This information contains the value, state, scope, and type about the symbol.

A symbol table is simply a table which can be either linear, Tree or a hash table. It maintains an entry for each name in the following format:

```
<symbol name,  type,  attribute>
```

For example, if a symbol table has to store information about the following variable declaration:

```
static int interest;
```

then it stores the entry as follow:

```
<interest, int, static>
```

Implementation of Symbol Table

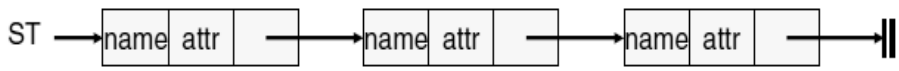
If a compiler is to handle a small amount of data, then the symbol table can be implemented as an unordered list, which is easy to code, but it is only suitable for small tables. A symbol table can be implemented in one of the following ways:

- Linear (sorted or unsorted) list (Linked List)
- Binary Search Tree
- Hash table

Among all, symbol tables are mostly implemented as hash tables, where the source code symbol itself is treated as a key for the hash function and the return value is the information about the symbol.

1. Linear (sorted or unsorted) list (Linked List)

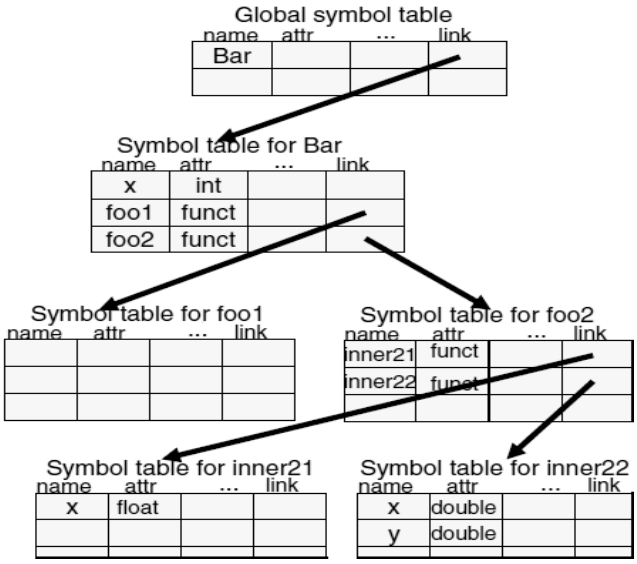
i. In this method, a **linked list** is used to store names and associated information.
ii. A pointer "**available**" is maintained at end of all stored records and new names are added in the order as they arrive
iii. To search for a name we start from beginning of list till available pointer and if not found we get an error "**use of undeclared name**"
iv. While inserting a new name we must ensure that it is not already present otherwise error occurs i.e. "**Multiple defined name**"
v. Insertion is fast, but lookup is slow for large tables
vi. Advantage is that it takes minimum amount of space
vii. Insertion is fast as **O(1)**, but lookup is slow for large tables – **O(n)** on average



2. Binary Search Tree

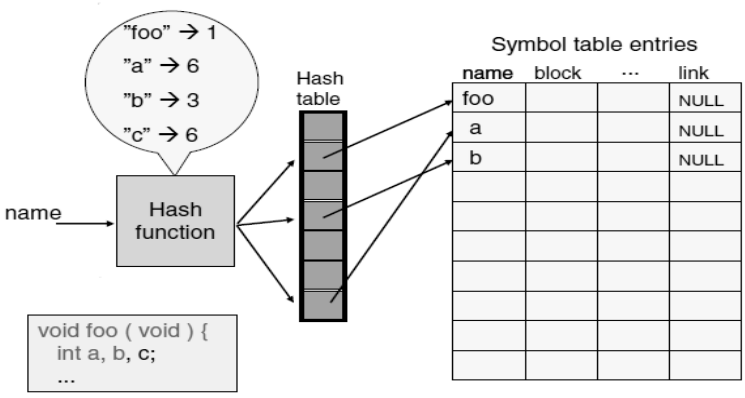
i. Another approach to implement symbol table is to use binary search tree i.e. we add two link fields i.e. left and right child.
ii. All names are created as child of root node that always follows the property of binary search tree.
iii. Insertion and lookup are **O($\log_2$ n)** on average

```
class Bar {
   int x;
   void foo1( ... ) { ... }
   void foo2( ... ) {
        int inner21( ... ) {
           float x;
           ...
        }
        int inner22( ... ) {
           double x, y;
           ...
           foo1( x );
        }
        ...
    }
    ...
}
...
```



3. Hash table

i. In hashing scheme two tables are maintained – a **hash table** and **symbol table** and is the most commonly used method to implement symbol tables.
ii. A hash table is an array with index range: **0** to **tablesize – 1**.These entries are pointer pointing to names of symbol table.
iii. To search for a name we use hash function that will result in any integer between **0** to **tablesize – 1**.
iv. Insertion and lookup can be made very fast
v. Advantage is quick search is possible and disadvantage is that hashing is complicated to implement.
vi. Insertion and lookup can be made very fast – **O(1)**.



Operations can be applied on Symbol Table

A symbol table can be implemented by either linear, binary search, or hash. These implementations must provide the following operations.

1. **Insertion**
2. **Lookup**

1. Insertion operation: insert()

This operation is more frequently used by analysis phase, i.e., the first half of the compiler where tokens are identified and names are stored in the table. This operation is used to add information in the symbol table about unique names occurring in the source code. The format or structure in which the names are stored depends upon the compiler in hand.

An attribute for a symbol in the source code is the information associated with that symbol. This information contains the value, state, scope, and type about the symbol. The insert() function takes the symbol and its attributes as arguments and stores the information in the symbol table.

For example:

```
int a;
```

should be processed by the compiler as:

```
insert(a, int);
```

2. Lookup operation: lookup()

lookup() operation is used to search a name in the symbol table to determine:

- Whether the symbol exists in the table or not.
- Whether it is declared before it is being used or not.
- Whether the name is used in the scope or not.
- Whether the symbol is initialized or not.
- Whether the symbol declared multiple times or not.

The format of lookup() function varies according to the programming language. The basic format should match the following:

```
lookup(symbol)
```

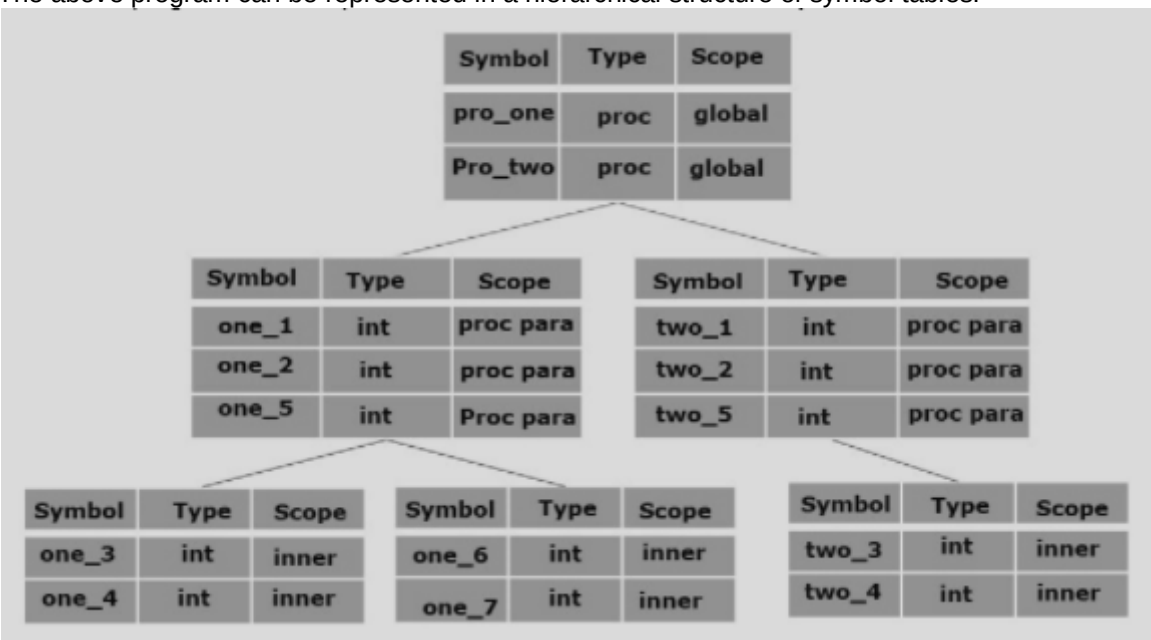
This method returns 0 (zero) if the symbol does not exist in the symbol table. If the symbol exists in the symbol table, it returns its attributes stored in the table.

Scope Management

A compiler maintains two types of symbol tables: a **global symbol table** which can be accessed by all the procedures and **scope symbol tables** that are created for each scope in the program. To determine the scope of a name, symbol tables are arranged in hierarchical structure as shown in the example below:

```
. . .
int value=10;

void pro_one()
   {
   int one_1;
   int one_2;

      {                \
      int one_3;        |_  inner scope 1
      int one_4;        |
      }                /

   int one_5;

      {                \
      int one_6;        |_  inner scope 2
      int one_7;        |
      }                /
   }

void pro_two()
   {
   int two_1;
   int two_2;

      {                \
      int two_3;        |_  inner scope 3
      int two_4;        |
      }                /

   int two_5;
   }
. . .
```

The above program can be represented in a hierarchical structure of symbol tables:



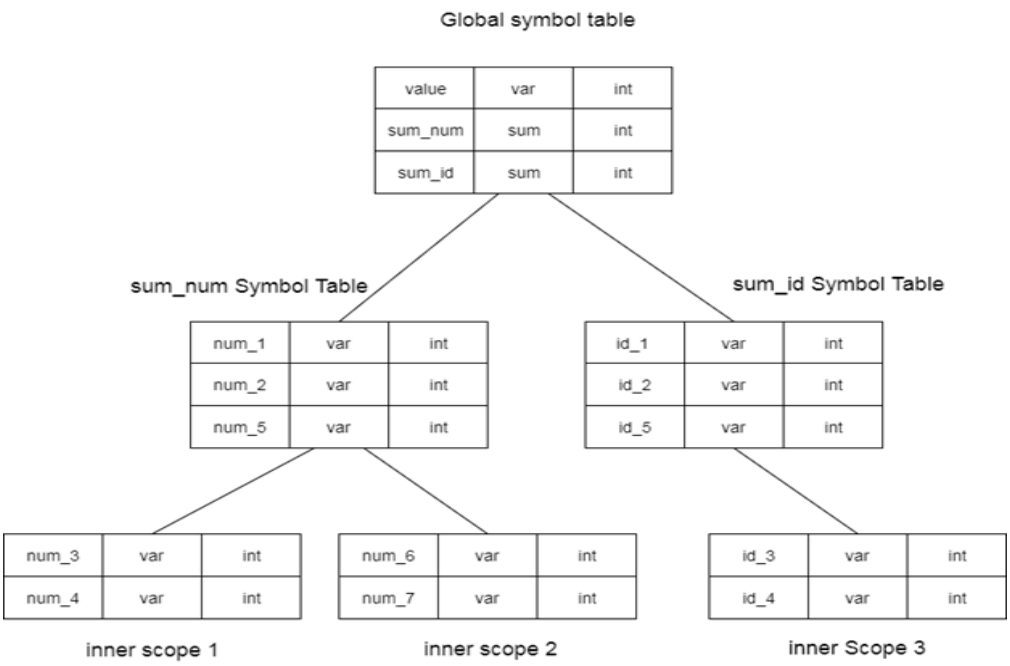
The global symbol table contains names for one global variable (int value) and two procedure names, which should be available to all the child nodes shown above. The names mentioned in the

pro_one symbol table (and all its child tables) are not available for pro_two symbols and its child tables.
This symbol table data structure hierarchy is stored in the semantic analyzer and whenever a name needs to be searched in a symbol table, it is searched using the following algorithm:

- first a symbol will be searched in the current scope, i.e. current symbol table.
- if a name is found, then search is completed, else it will be searched in the parent symbol table until,
- either the name is found or global symbol table has been searched for the name.

Another Example:

```
int value=10;
void sum_num()
   {
     int num_1;
     int num_2;
         {
         int num_3;
         int num_4;
         }
      int num_5;

         {
          int_num 6;
          int_num 7;
         }
   }
Void sum_id
   {
     int id_1;
     int id_2;
         {
         int id_3;
         int id_4;
         }
      int num_5;
   }
```



Runtime Environments

We need to come up with a representation of these high-level structures using the low-level structures of the machine.
A runtime environment is a set of data structures maintained at runtime to implement these high-level structures. e.g. the stack, the heap, static area, virtual function tables, etc.

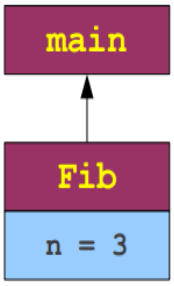
Activation Trees

A program consists of procedures, procedure declaration, procedure definition, procedure name. Each execution of procedure is referred to as an activation of the procedure. Lifetime of activation is the sequence of steps present in the execution of the procedure.

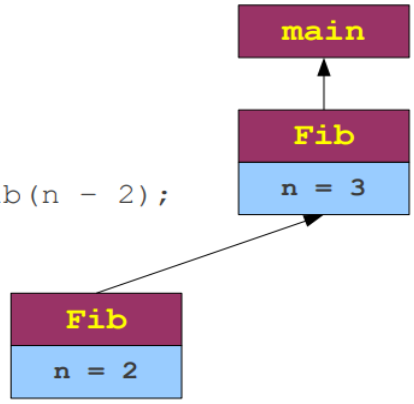
i. An activation tree is a tree structure representing all of the function calls made by a program on a particular execution.
ii. It depends on the runtime behavior of a program; can't always be determined at compile-time.
iii. The root shows the activation of the main function.
iv. Each node in the tree is an activation record.
v. Each activation record stores a control link to the activation record of the function that invoked it.
vi. The node for procedure ‘x’ is the parent of node for procedure ‘y’ if and only if the control flows from procedure x to procedure y.

```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```

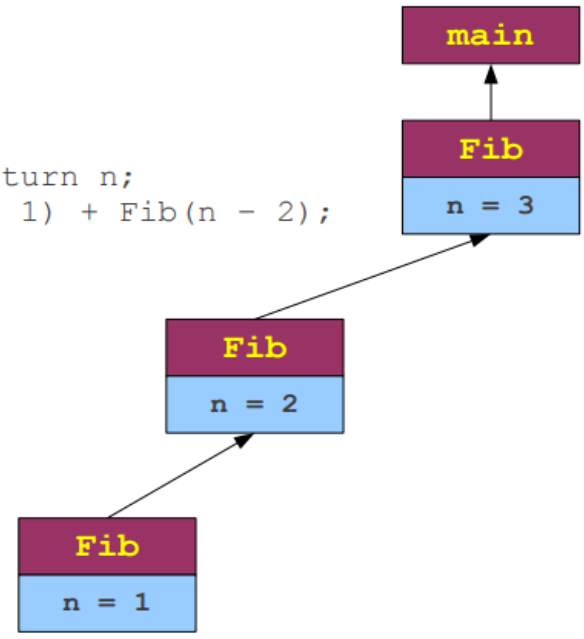
```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```



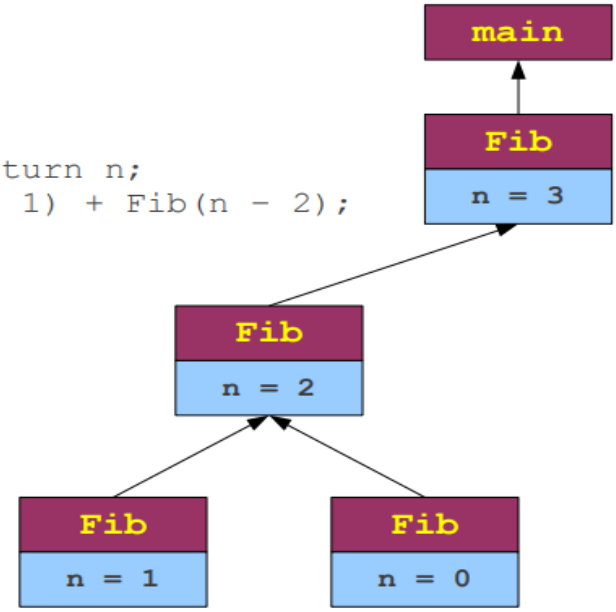
```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```



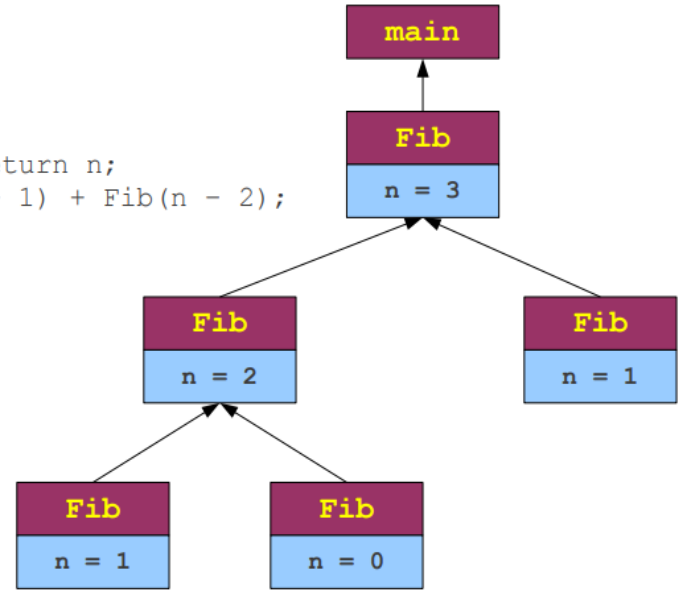

```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```




```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```



```
int main() {
    Fib(3);
}

int Fib(int n) {
    if (n <= 1) return n;
    return Fib(n - 1) + Fib(n - 2);
}
```



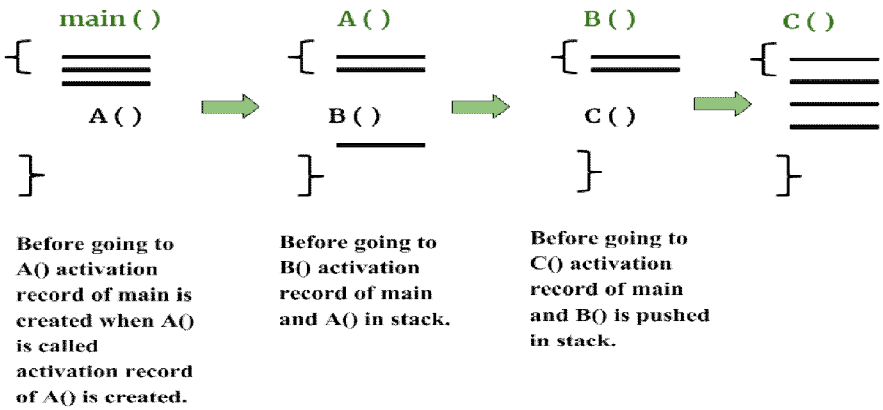
CONTROL STACK AND ACTIVATION RECORDS

Function calls are often implemented using a stack of activation records.

i. Calling a function pushes a new activation record onto the stack.

ii. Returning from a function pops the current activation record from the stack.

Control stack or runtime stack is used to keep track of the live procedure activations i.e. the procedures whose execution have not been completed. A procedure name is pushed on to the stack when it is called (activation begins) and it is popped when it returns (activation ends). Information needed by a single execution of a procedure is managed using an activation record or frame. When a procedure is called, an activation record is pushed into the stack and as soon as the control returns to the caller function the activation record is popped.



A general activation record consists of the following things:

1. **Local variables:** hold the data that is local to the execution of the procedure.
2. **Temporary values**: stores the values that arise in the evaluation of an expression.
3. **Machine status:** holds the information about status of machine just before the function call.
4. **Access link (optional):** refers to non-local data held in other activation records.
5. **Control link (optional):** points to activation record of caller.
6. **Return value:** used by the called procedure to return a value to calling procedure
7. **Actual parameters**

RUNTIME MEMORY MANAGEMENT

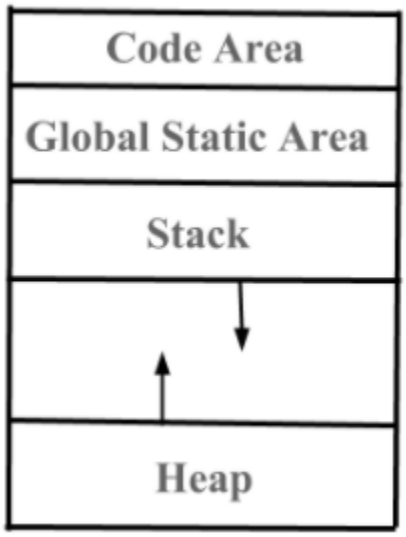
Runtime storage subdivides the memory to hold:

i. Target code- the program code , it is static as its size can be determined at compile time

ii. Static data objects

iii. Dynamic data objects- heap

iv. Automatic data objects- stack



STORAGE ALLOCATION TECHNIQUES

I. Static Storage Allocation

For any program if we create memory at compile time, memory will be created in the static area. For any program if we create memory at compile time only, memory is created only once. It don't support dynamic data structure i.e memory is created at compile time and deallocated after program completion. The drawback with static storage allocation is recursion is not supported. Another drawback is size of data should be known at compile time

Eg- FORTRAN was designed to permit static storage allocation.

II. Stack Storage Allocation

Storage is organised as a stack and activation records are pushed and popped as activation begin and end respectively. Recursion is supported in stack allocation

III. Heap Storage Allocation

Memory allocation and deallocation can be done at any time and at any place depending on the requirement of the user. Heap allocation is used to dynamically allocate memory to the variables and claim it back when the variables are no more required. Recursion is supported.

PARAMETER PASSING

The communication medium among procedures is known as parameter passing. The values of the variables from a calling procedure are transferred to the called procedure by some mechanism.

Basic terminology:

R-value: The value of an expression is called its r-value. The value contained in a single variable also becomes an r-value if it appears on the right side of the assignment operator. R-value can always be assigned to some other variable.
L-value: The location of the memory (address) where the expression is stored is known as the l-value of that expression. It always appears on the left side of the assignment operator.
Formal Parameter: Variables that take the information passed by the caller procedure are called formal parameters. These variables are declared in the definition of the called function.
Actual Parameter: Variables whose values and functions are passed to the called function are called actual parameters. These variables are specified in the function call as arguments.

Different ways of passing the parameters to the procedure:

i. **Call by Value**
ii. **Call by Address**
iii. **Call by Reference**